Machine Learning and Optimization — Problem Set 2

Problem 1: Ridge regression and cross-validation

1. Feature scaling

The features have very different scales: career totals can be thousands, annual counts are smaller, and indicator variables are 0 or 1. Ridge penalizes coefficient magnitudes, which depend on the measurement units. Without scaling, a large-scale feature needs a smaller coefficient for the same predictive effect and is consequently penalized less. Standardizing makes shrinkage comparable across features.

For each column j, compute

$$\mu_j = \frac{1}{n}\sum_{i=1}^{n} X_{ij}, \qquad s_j = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(X_{ij}-\mu_j)^2}, \qquad Z_{ij} = \frac{X_{ij}-\mu_j}{s_j}.$$

Each nonconstant column then has mean zero and standard deviation one (`ddof=0`). During cross-validation, compute these statistics from the training fold only and apply them to validation data. Do not standardize the intercept or target salaries.

2. Closed-form solution with an unpenalized intercept

Let $Z \in \mathbb{R}^{n\times p}$ contain the standardized features. Define

$$A = [\mathbf{1}\ Z], \quad \boldsymbol{\beta} = \begin{bmatrix} b \\ \boldsymbol{\theta} \end{bmatrix}, \quad D = \operatorname{diag}(0, 1, \ldots, 1).$$

We minimize

$$J(\boldsymbol{\beta}) = \|\mathbf{y} - A\boldsymbol{\beta}\|_2^2 + \lambda \boldsymbol{\beta}^\mathsf{T} D \boldsymbol{\beta}.$$

Since $\boldsymbol{\beta}^\mathsf{T} D \boldsymbol{\beta} = \sum_{j=1}^{p} \theta_j^2$, the intercept b is not penalized. The gradient is

$$\nabla J = -2A^\mathsf{T}(\mathbf{y} - A\boldsymbol{\beta}) + 2\lambda D\boldsymbol{\beta}.$$

Setting it to zero gives

$$(A^\mathsf{T}A + \lambda D)\hat{\boldsymbol{\beta}} = A^\mathsf{T}\mathbf{y},$$

$$\boxed{\hat{\boldsymbol{\beta}} = (A^\mathsf{T}A + \lambda D)^{-1}A^\mathsf{T}\mathbf{y}}.$$

Here A is the assignment's augmented matrix $\tilde{X} = [\mathbf{1}\ X]$ after feature standardization. The implementation solves the linear system without explicitly forming an inverse. For $\lambda > 0$ this system has a unique solution.

Since the columns of Z have zero means,

$$\hat{b} = \bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i, \qquad \hat{\boldsymbol{\theta}} = (Z^\mathsf{T}Z + \lambda I)^{-1} Z^\mathsf{T}(\mathbf{y} - \bar{y}\mathbf{1}).$$

Our objective uses a sum of squared errors. For the alternative objective

$$\frac{1}{n}\|\mathbf{y} - A\boldsymbol{\beta}\|_2^2 + \alpha\|\boldsymbol{\theta}\|_2^2, \qquad \alpha = \frac{\lambda}{n}.$$

3. Coefficient norm along the regularization path

The grid is $\lambda_j = 10^{-3+10j/99}$, $j = 0, \ldots, 99$. The first plot shows $\|\hat{\boldsymbol{\theta}}\|_2$ against λ with logarithmic axes; the intercept is excluded. All 99 successive differences in the norm are nonpositive within numerical tolerance. This monotonicity also follows from optimality. If $0 < \lambda_1 < \lambda_2$ and $\boldsymbol{\theta}_1, \boldsymbol{\theta}_2$ are the respective minimizers, adding their two optimality inequalities gives $(\lambda_2 - \lambda_1)(\|\boldsymbol{\theta}_2\|_2^2 - \|\boldsymbol{\theta}_1\|_2^2) \leq 0$. Thus the norm cannot increase as λ increases. The objective penalizes the squared norm; the requested plot displays the norm itself.

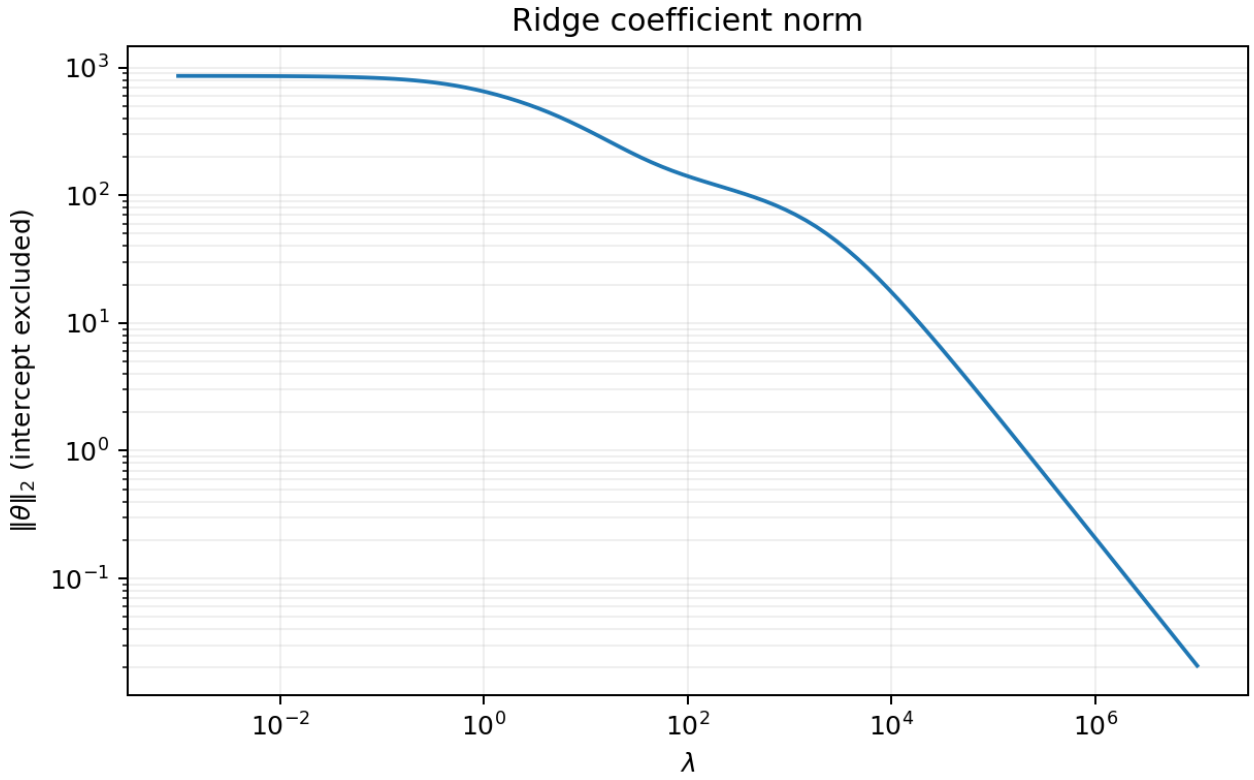


Figure 1: Coefficient norm

4. Small- and large-lambda limits

At $\lambda = 10^{-8}$, the Euclidean difference between the augmented ridge and least-squares coefficients is 5.256987×10^{-6}, a relative difference of 5.1798283×10^{-9}. Least squares is computed independently using numpy.linalg.lstsq. At the required grid endpoint $\lambda = 10^{-3}$, the absolute coefficient difference is 0.524964; the still smaller lambda demonstrates the limiting behavior more clearly. At $\lambda = 10^{12}$, $\|\hat{\boldsymbol{\theta}}\|_2 = 2.0720445 \times 10^{-7}$, while $\hat{b} = 535.92584411$. At the grid endpoint $\lambda = 10^7$, the slope norm is 0.02071657. Thus all slopes approach zero and the prediction approaches the constant mean salary, rather than zero. In the centered formula, the eigenvalues of $(Z^\top Z + \lambda I)^{-1}$ go to zero as λ grows, proving the result.

5. Manual five-fold cross-validation

Use numpy.random.default_rng(42) to permute row indices, then numpy.array_split to create disjoint folds of sizes 53, 53, 53, 52, 52. For each lambda and fold, fit standardization on the other four folds, solve the ridge system on those training observations, and predict the held-out observations. No held-out

target or feature statistic is used in fitting. The score is pooled out-of-fold mean squared error:

$$\mathrm{CV}(\lambda) = \frac{1}{263} \sum_{i=1}^{263} \left(y_i - \hat{y}_i^{(-k(i))}(\lambda) \right)^2 .$$

Here $k(i)$ is the fold containing observation i, and the prediction is fitted without that fold. This weights folds by their sizes. Choose the grid point minimizing this score, then refit on all observations with full-data scaling.

The selected value and errors are

$$\lambda^* = 2.15443469, \qquad \mathrm{CV}(\lambda^*) = 114713.077893,$$

$$\mathrm{RMSE} = \sqrt{\mathrm{CV}(\lambda^*)} = 338.693191.$$

MSE is measured in squared thousands of dollars; RMSE is in thousands of dollars. A different partition or scaling convention can select a different grid point. This is a tuning score, not an unbiased independent test-set estimate after model selection; no separate test set is provided.

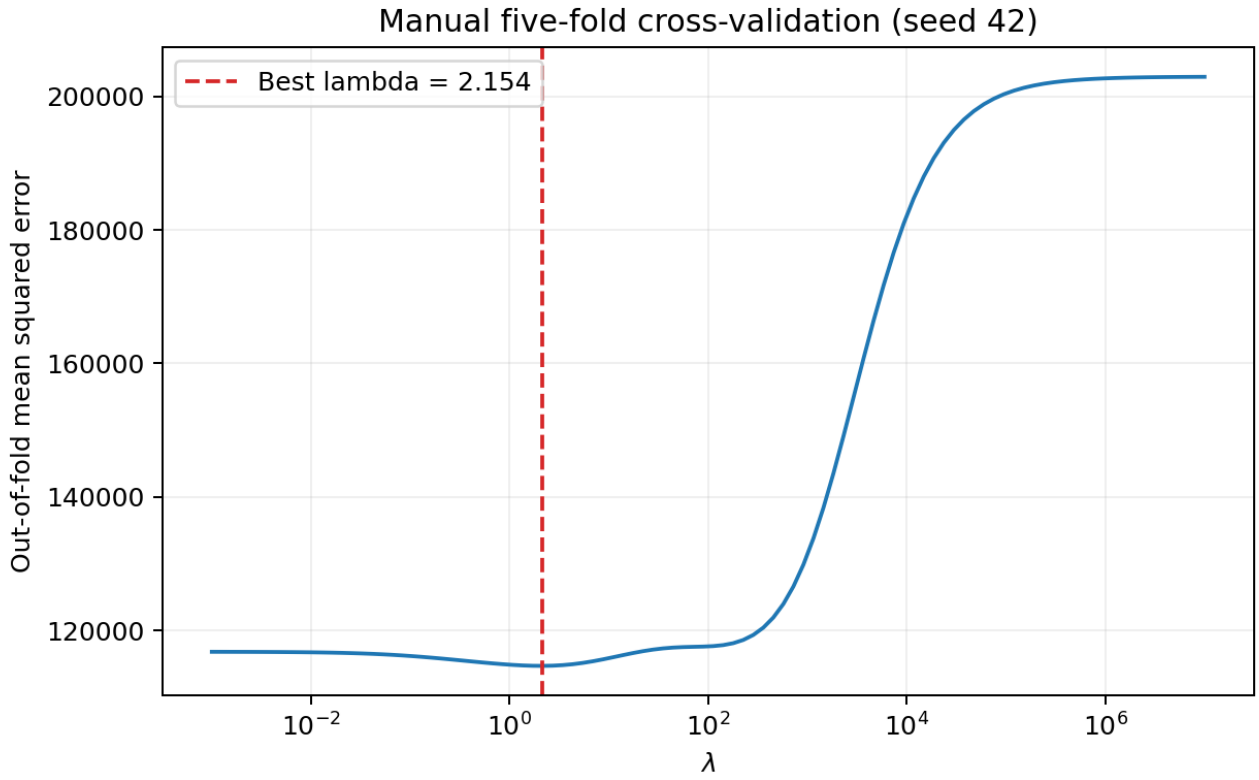


Figure 2: Cross-validation curve

6. Final coefficients and coaching interpretation

The table below gives the refitted coefficients for standardized features and for original feature units. For a standardized feature, the coefficient is the fitted salary difference associated with a one-standard-deviation increase while holding other predictors fixed. For a binary feature, its original-units coefficient

describes the full 0-to-1 change. The standardized intercept is the fitted salary at mean feature values. The original-units coefficients are

$$\hat{\theta}_j^{\text{orig}} = \frac{\hat{\theta}_j}{s_j}, \qquad \hat{b}^{\text{orig}} = \hat{b} - \sum_{j=1}^{p} \frac{\mu_j \hat{\theta}_j}{s_j}.$$

Both parameterizations give identical predictions.

Feature	Standardized coefficient	Original-units coefficient
Intercept	535.925844	151.014125
AtBat	-243.242974	-1.654411
Hits	261.711860	5.810724
HmRun	8.250934	0.943995
Runs	-12.058313	-0.473038
RBI	-0.670097	-0.025939
Walks	115.188986	5.313947
Years	-47.736338	-9.977302
CAtBat	-142.652289	-0.062506
CHits	125.718010	0.194319
CHmRun	53.218355	0.648679
CRuns	245.959348	0.744050
CRBI	129.418440	0.400984
CWalks	-163.670003	-0.621013
LeagueN	30.657633	61.415236
DivisionW	-61.196818	-122.415761
PutOuts	78.146208	0.279691
Assists	42.883101	0.296145
Errors	-24.617609	-3.733333
NewLeagueN	-13.841256	-27.755036

A practical handful of attributes to emphasize would be: hitting reliably (Hits), producing runs over a sustained career (CRuns), reaching base through walks (Walks), and contributing defensively (PutOuts). Hits and CRuns have the largest positive standardized coefficients; Walks and PutOuts also have positive coefficients. Career hits, career RBI, and career home runs supply related evidence for sustained offensive production. These are predictive associations, not causal coaching effects. Annual and career measures are strongly related, so individual coefficients can change as correlated predictors share explanatory power. For example, the negative AtBat coefficient does not mean a player should avoid batting; conditional on the same number of hits and other statistics, additional at-bats can reflect weaker efficiency. Similarly, negative career-walk or years coefficients should not be interpreted as evidence that walking or experience reduces pay. Career totals are accumulated outcomes, not independently manipulable skills. League and division indicators are context rather than player attributes. Coaching advice should therefore focus on the broad skills above, without promising a salary increase from this model.

Reproduction

Run `python solution.py` from this directory (or use its absolute path). Required runtime dependencies: NumPy and Matplotlib.